

Idle Kingdom Clicker Oyun Matematiği Analizi

<https://gemini.google.com/app/9cf05a095ea1e71a>

User prompt: idle kingdom clicker oyununun tüm matematiğini analiz et, ne nasıl artıyor, ne neye bağlı, seviye arttırmaları katsayılar vs. bir oyun dizaynı gibi tüm bu sistemi detaylıca açıkla

Response: Bir oyun tasarımcısı şapkasıyla "Idle Kingdom Clicker"ın kaputunun altına baktığımızda, oyunun basit bir tıklama mekaniğinden ziyade karmaşık bir **ekonomi dengeleme motoru (economy balancing engine)** olduğunu görürüz. Oyun, geleneksel üstel büyüme formüllerini (exponential scaling), altıgen (hex) harita tabanlı uzamsal çarpanlarla harmanlayarak bağımlılık yapıcı bir döngü yaratır.

İşte bu sistemin tüm matematiği ve tasarım mimarisi:

1. Temel Döngü: Üretim ve Maliyet Tahtereallisi

Tüm idle oyunlarının kalbinde, oyuncunun ilerlemesini kontrol altında tutan "üstel maliyet artışı" yatar. Oyuncu yeni bir üretim tesisi aldığında veya seviyesini yükselttiğinde maliyetler katlanarak artarken, üretim daha doğrusal (linear) bir ivmeyle büyür. Bu durum oyuncuyu er ya da geç bir "gelişim duvarına" çarptırır ve prestij yapmaya zorlar.

Bina Maliyeti Formülü (Exponential Cost):

$$Maliyet_n = TemelMaliyet \times (BüyümeKatsayısı)^n$$

- n:** Binanın mevcut seviyesi.
- Büyüme Katsayısı:** Oyun tasarımında bu değer genellikle 1.07 ile 1.15 arasındadır. Idle Kingdom Clicker'da giriş seviyesi üretim binaları için bu katsayı düşük tutulurken, oyunun sonlarındaki gelişmiş üretim tesisleri (tier 3 ürünler) için bu katsayı daha yüksektir.

Üretim Formülü (Multiplicative Growth): Üretim artışı seviye ile doğrudan bağlantılıdır, ancak asıl sıçramayı çarpanlar (multipliers) yapar.

$$Üretim = (TemelÜretim \times Seviye) \times (Harita \text{ Çarpanları} \times Yetenek \text{ Çarpanları})$$

Tasarımcı Notu: Bu simülatörde görebileceğiniz gibi, "Büyüme Katsayısı" ufak bir miktar bile artsa, birkaç seviye sonra maliyet çizgisi dikey bir duvara dönüşür. Idle Kingdom Clicker, bu duvarı aşmanız için size "Harita Çarpanları"nı verir.

2. Uzamsal Çarpanlar (Spatial Multipliers)

Oyunun geleneksel idle oyunlarından ayrıştığı ana nokta altıgen haritasıdır. Idle Kingdom Clicker'da üretim, sadece binanın seviyesine değil, aynı zamanda lojistik ağına ve haritadaki konumuna (Adjacency Bonus) bağlıdır.



- Bölgesel Bonuslar (Tile Multipliers):** Haritadaki "Midland" gibi belirli özel karolar, etrafındaki binalara devasa statik bonuslar verir. Örneğin; yemek üretim karoları doğrudan **6x**, belirli tarım noktaları ise **12x**'e kadar çiftlik çarpanı (farm multiplier) sağlar.
- Lojistik ve Taşımacılık:** Haritada kurduğunuz yollar ve taşımacılık ağları (transportation), üretim binalarını birbirine bağlayarak etki alanındaki binalara ek **3x3** çarpanlar yaratır.

3. Üretim Zinciri (Supply Chain) Dinamikleri

Oyunda 20 civarı hammadde (raw material) ve 30'dan fazla işlenmiş ürün (finished product) bulunur. Bir ürün ağacı tasarımı olarak oyuncu sürekli şu karar mekanizmasıyla test edilir:

$$Kâr = (SatışFiyatı \times SatışHızıÇarpanı) - (\Sigma GirdiMaliyetleri)$$

Tasarımcılar, hammaddeleri satmanın getirisini bilinçli olarak düşük tutar. Oyuncu, kazancını maksimize etmek için birden fazla hammaddeyi birleştiren fabrikalar kurmak zorundadır. Ancak işlenmiş ürün binalarının satın alma maliyet katsayısı daha yüksektir. Bu da "yüksek kâr marjı = yüksek yatırım maliyeti" dengesini kurar.

4. Prestij Sistemi (Zamanı Sıfırlama)

Maliyet eğrisi artık üretim eğrinizi fersah fersah geçtiğinde, altın kazanmak için bekleme süreniz günlere çıkar. Bu noktada oyunu hızlandırmak için Prestij (veya Soft Reset) devreye girer.

Prestij formülü şu mantıkla çalışır:

$$KazanılacakPuan = \left(\frac{ToplamKazanılanAltın}{SabitÖlçek} \right)^{0.5}$$

Bu sistemdeki **0.5 üssü** (karekök) kritik bir oyun tasarımı hilesidir. Eğer önceki denemenizde 1 Milyon altın kazanıp 1000 prestij puanı aldıysanız, bir sonraki sefer 2000 puan alabilmek için 2 Milyon değil, tam 4 Milyon altın kazanmanız gerekir (Azalan Verim - Diminishing Returns).

Idle Kingdom Clicker, bu puanları harcayabilmeniz için oyuncuya **Beş farklı Göksel Yetenek Ağacı (Celestial Skill Tree)** sunar. Bu ağaçlardan kalıcı olarak Global Üretim Çarpanı veya görev otomasyonları alırsınız. Bu sayede bir sonraki turda maliyet duvarına çarpmanız çok daha uzun sürer.

Sources

1. [The Math of Idle Games, Part I - Game Developer](#)
2. [Midland - Idle Kingdom Clicker Wiki - Fandom](#)
3. [Small reviews of \(I think\) all incremental games I've ever played on Android - Reddit](#)
4. [Inventory Idle - Aplikacije na Google Playu](#)

User prompt: Idle oyunlarındaki prestij formüllerinde karekök veya logaritmik hesaplamalar oyunun ilerleme hızını tam olarak nasıl etkiler?

Response: Idle oyunlarında prestij formülleri, oyunun ritmini (pacing) ve oyuncunun psikolojisini belirleyen en güçlü araçlardır. Oyunun temelindeki kaynak üretimi üstel (exponential) olarak artarken, prestij puanı kazanımı **kasten yavaşlatılarak** (diminishing returns - azalan verim) dengelenir.

Karekök ve logaritmik formüllerin her ikisi de oyuncunun ilerleyişini frenler, ancak bunu yapış biçimleri oyunun döngüsünü tamamen değiştirir.

Bir oyun tasarımcısı olarak bu iki yaklaşımı ve ilerleme hızına etkilerini şöyle ayırabiliriz:

1. Karekök (Square Root) Büyüme: "Biraz Daha İleri Git" Döngüsü

Karekök tabanlı sistemler (örneğin $Puan = \sqrt{KazanılanAltın}$), oyuncuya tatmin edici ama giderek zorlaşan bir hedef sunar.

- **Matematiksel Etkisi:** Prestij puanınızı **2 katına** çıkarmak istiyorsanız, mevcut kaynağınızın **4 katını** ($\rightarrow 2^2$) toplammanız gerekir. 3 katı puan için **9 katı** kaynak gerekir.
- **Oyun İçi Hissiyatı:** Oyuncu mevcut prestij turunda biraz daha beklerse, gözle görülür bir kazanç elde edeceğini hisseder. "Duvar" yavaş yavaş kalınlaşır.
- **İlerleme Hızı:** Orta ve uzun süreli döngüler (runs) yaratır. Oyuncu prestij yapmadan önce birkaç saat veya birkaç gün bekleyerek ekstra puan koparmak için "push" (zorlama) yapabilir.

2. Logaritmik Büyüme: "Hemen Sıfırla" Döngüsü

Logaritmik sistemler (örneğin $Puan = \log_{10}(Kazanılan Altın)$), oyuncunun karşısına aşılması imkansız, sert bir beton duvar örer.

- Matematiksel Etkisi:** İlk 1 prestij puanını 10 altınla aldıysanız, 2. puan için 100, 3. puan için 1.000, 4. puan için 10.000 altın gerekir. Kaynak ihtiyacı üstel olarak (10^n 'un kuvvetleri şeklinde) artar.
- Oyun İçi Hissiyatı:** Oyuncu belirli bir noktaya çok hızlı ulaşır, ancak o noktadan sonra bir sonraki prestij puanını almak için geçmesi gereken süre mantıksız derecede uzundur (günlerden yıllara çıkabilir).
- İlerleme Hızı:** Çok kısa ve agresif döngüler yaratır. Oyuncu "duvara" çarptığı an (üretim hızı yavaşladığı an) beklemek anlamsızlaşır. İlerlemenin tek yolu derhal prestij yapıp baştan başlamaktır. *Antimatter Dimensions* gibi devasa sayılarla (BigNumber) çalışan oyunlar bu sistemi kullanır.

Aşağıdaki araç ile bu iki formülün yüksek kaynak miktarlarında nasıl davrandığını (ve nasıl düzleştirdiğini) görselleştirebilirsiniz:

Tasarımcı Notu: Aracın içindeki grafikte x eksenini (kazanılan kaynak) büyüttüğünüzde, logaritmik çizginin neredeyse tamamen yatay bir çizgiye dönüştüğünü göreceksiniz. İşte bu yatay çizgi, oyuncunun oyunu sıfırlayıp baştan başlamak zorunda olduğu o kırılma anıdır.

Hangi Formül Ne Zaman Kullanılmalı?

Karşılaştırma	Karekök (Square Root)	Logaritma (Logarithmic)
İdeal Kullanım Yeri	Altın, para veya eşya gibi nispeten yavaş artan doğrusal/multiplikatif kaynaklar.	Saniyede milyarlarca artan hiper-üstel üretim (ör. evrenler, galaksiler).
Oyuncu Davranışı	"Biraz daha bekleyip 10 puan yerine 15 puan alayım."	"Üretimim durma noktasına geldi, hemen prestij yapmalıyım."
Tasarım Riski	Prestij yetenekleri çok güçlüyse kartopu etkisi (snowballing) kontrolden çıkabilir.	Erken safhalarda oyuncuyu çok sık prestij yapmaya zorlayarak yorabilir.

User prompt: oyundaki yapı ve ürün çeşitliliğini ve birbirleri arasındaki bağlamları açıkla

Response: *Idle Kingdom Clicker*'in ekonomisi, basit bir tıklama oyunundan çok, mikro yönetim ve lojistik gerektiren bir tedarik zinciri (supply chain) simülasyonu gibidir. Oyundaki yapıların ve ürünlerin çeşitliliği, basit hammaddelerden (Tier 1) başlayıp, onları karmaşık ve kârlı işlenmiş ürünlere (Tier 2 ve Tier 3) dönüştüren bir ağacın dalları şeklinde tasarlanmıştır.

Bu sistemin en kritik yönü, ürünleri üretmenin ve bağlamanın tek başına yeterli olmaması; aynı zamanda haritadaki **uzamsal konumlandırmanın (spatial adjacency)** üretim verimliliğini doğrudan etkilemesidir.

Oyun içindeki yapı hiyerarşisi ve ürünlerin birbirine nasıl bağlandığı aşağıda detaylı olarak açıklanmıştır:

1. Üretim Binaları ve Ürün Seviyeleri (Tiers)

Binalar, ürettikleri ürünün karmaşıklığına göre üç ana seviyeye ayrılır. İlerledikçe, alt seviye ürünler üst seviye binaların girdisi (input) haline gelir.

Seviye	Bina Örneği	Ürün (Çıktı)	Girdi Gereksinimi	Bağlam ve Tasarım Amacı
Tier 1 (Hammadde)	Oduncu (Woodcutter)	Odun (Wood)	Yok (Bağımsız)	Doğrudan doğadan toplanan, kâr marjı çok düşük olan temel yapı taşları.
	Çiftlik (Farm)	Buğday (Wheat)	Yok (Bağımsız)	Oyunun başında temel geliri sağlarlar.

Seviye	Bina Örneği	Ürün (Çıktı)	Girdi Gereksinimi	Bağlam ve Tasarım Amacı
Tier 2 (İşlenmiş)	Kereste Fabrikası (Sawmill)	Kereste (Planks)	Odun (Wood)	Odunun değerini katlayarak satılmasını sağlar. İlk stratejik kararı gerektirir: Hammaddeyi mi satayım, işleyeyim mi?
	Değirmen (Windmill)	Un (Flour)	Buğday (Wheat)	Kâr marjı orta seviyededir. Lojistik (yol) bağlantısı zorunludur.
Tier 3 (Karmaşık)	Mobilyacı (Furniture Maker)	Mobilya (Furniture)	Kereste + Demir	Oyundaki en yüksek kâr marjına sahip, birden fazla farklı Tier 2 (veya 1) ürünün birleşimini gerektiren "end-game" üretimleri.
	Fırın (Bakery)	Ekmek (Bread)	Un + Su	Kompleks lojistik ağlar kurmayı zorunlu kılar.

2. Altıgen (Hex) Harita ve Lojistik Bağlamı

Ürünlerin birbirleriyle olan ilişkisi sadece envanter (inventory) üzerinden yürümez. Bir Tier 2 binasının üretime geçebilmesi için, ihtiyaç duyduğu Tier 1 malzemesine **fiziksel olarak bağlı** olması gerekir.

- Taşıma Yolları (Transport Routes):** Bir Değirmen'in (Un üretmek için) Buğdaya ihtiyacı vardır. Eğer Çiftlik ile Değirmen arasında altıgen haritada bir "Yol" bağlantısı yoksa, Değirmen üretim yapamaz. Bu durum, oyuncuyu haritayı optimize etmeye (spagetti gibi yollar çizmekten kaçınmaya) zorlar.
- Kümelenme (Clustering) Bonusu:** Belirli binalar, yan yana dizildiklerinde ekstra çarpan kazanır. Örneğin, Oduncu kamplarını orman karolarının (tile) etrafına dairesel olarak yerleştirmek, her birinin üretim hızını (adjacency bonus) ciddi şekilde artırır.
- Depolama ve Şehir Merkezi (Town Hall):** Tüm ürünlerin nihai olarak satılabilmesi veya altın/prestij puanına dönüşebilmesi için Şehir Merkezine bağlı bir yol ağında olması gerekir. Şehir Merkezi, tüm bu tedarik zincirinin nihai boşaltım (sink) noktasıdır.

3. Çarpan (Multiplier) ve Bağımlılık Dinamiği

Bu çok katmanlı üretim ağacında tasarımcıların oyuncuyu kilitlediği bir matematiksel kural vardır: **Girdi maliyeti (Input Cost) her zaman sabittir, ancak üretim kapasiteleri binaların seviyesi (level) arttıkça bozulur.**

- Eğer Çiftliğinizi 50. seviyeye, Değirmeninizi ise 10. seviyeye yükseltirseniz; Çiftlik saniyede 100 Buğday üretirken, Değirmen saniyede sadece 10 Buğdayı Un'a çevirebilir.
- Bu darboğazı (bottleneck) aşmak için oyuncu ya yeni Değirmenler inşa etmeli (ki maliyet katsayısı üstel olarak artar) ya da Global Çarpanları (Prestij yetenekleri) devreye sokmalıdır.
- Tasarımsal Sonuç:** Oyuncu hiçbir zaman tek bir binayı sonsuza kadar yükseltip oyunu kıramaz (break the game). Sistemin kendi içindeki girdi-çıkıtlı bağımlılığı, oyuncuyu tüm ekonomiyi dengeli bir şekilde büyütmeye (balance the economy) zorlar.

User prompt: Bu girdi-çıkıtlı hiyerarşisini ve ürün bağımlılıklarını Godot'da GDScript kullanarak veri kaynakları (Resources) veya Dictionary yapısı ile nasıl modelleyebilirim?

Response: Godot 4'te Idle Kingdom Clicker benzeri bir üretim ekonomisini kurarken, verileri Dictionary yerine kesinlikle **Custom Resources (Özel Veri Kaynakları)** kullanarak modellemelisiniz.

Antigravity IDE ve Hermes Agent üzerinde kurduğunuz o 6 alt-ajanlı sistem mimarisini düşünürseniz; ajanlarınızın (veya oyun motorunun) verileri statik bir JSON veya devasa bir Dictionary içinde aramasındansa, birbirine referans veren, tip güvenli (type-safe) Godot Kaynaklarını okuması hata payını sıfıra indirir.

Ayrıca Resources kullandığınızda, Godot'nun Inspector panelini bir veritabanı yönetim arayüzü gibi kullanabilirsiniz.

İşte bu mimariyi GDScript ile kurmak için ihtiyacınız olan üç temel Resource sınıfı:

1. Temel Öğe: ItemData (Hammadde ve Ürünler)

Her odun, buğday veya ekmek bir ItemData kaynağıdır. Bu, eşyanın kimliğidir.

```
# item_data.gd
extends Resource
class_name ItemData

@export var item_id: String
@export var display_name: String
@export var icon: Texture2D
@export var base_value: float # Satış fiyatı
@export var tier: int = 1 # 1: Hammadde, 2: İşlenmiş, 3: Karmaşık
```

2. Üretim Reçetesi: RecipeData (Bağlamı Kuran Sınıf)

Bu sınıf, girdi ve çıktı bağımlılığını (dependency) yaratan köprüdür. Hangi eşyadan kaç adet gerektiğini tutmak için içeride küçük bir alt-sınıf (veya Dictionary) kullanabiliriz. Ancak tamamen Resource kalmak en temizidir.

```
# recipe_data.gd
extends Resource
class_name RecipeData

# Alt sınıf: Girdi miktarını ve eşyayı eşlemek için
class RecipeIngredient:
    var item: ItemData
    var amount: int

    func _init(_item: ItemData = null, _amount: int = 1):
        item = _item
        amount = _amount

@export var recipe_name: String
@export var production_time: float = 2.0 # Saniye cinsinden

# Inspector'da Array[Resource] olarak görünmesi için
@export var inputs: Array[ItemData]
@export var input_amounts: Array[int] # Sıralaması inputs ile aynı olmalı

@export var output_item: ItemData
@export var output_amount: int = 1
```

Not: Godot 4.3+ sürümlerinde Inspector dizilerini daha iç içe yönetmek için @export ile doğrudan custom struct'lar kullanılabilir, ancak yukarıdaki paralel dizi yöntemi prototipleme için çok hızlıdır.

3. Üretim Tesisleri: BuildingData

Binalar, reçeteleri barındıran kapsayıcılardır (Containers).

```
# building_data.gd
extends Resource
class_name BuildingData

@export var building_name: String
@export var base_cost: float = 100.0
@export var cost_growth_factor: float = 1.15 # Maliyet artış katsayısı

# Bu bina hangi üretim reçetelerini işleyebilir?
@export var available_recipes: Array[RecipeData]
```

Bu Mimari Neden Dictionary'den Daha İyi?

Eğer bu verileri Dictionary ile (örneğin bir JSON dosyasından) yapsaydınız: "inputs": {"wood": 2, "iron": 1} şeklinde string anahtarlar kullanacaktınız. Ürün ismini kodun bir yerinde yanlış yazdığınızda (typo) oyun

sessizce çökecekti.

Resource mimarisi ile:

1. Dosya sisteminizde (FileSystem) `res://data/items/wood.tres` diye bir dosya oluşturunuz.
2. Kereste Fabrikası (Sawmill) reçetesini oluştururken, "Girdiler" kısmına Inspector üzerinden doğrudan `wood.tres` dosyasını sürükleyip bırakırsınız.
3. Alt-ajanlarınıza (Hermes'teki promp'lara) "Oyunun eşya verileri `res://data` klasöründedir, `ItemData` tipindedir" dediğinizde, kod yazan ajanlar string eşleştirmesi yapmak zorunda kalmaz, doğrudan obje referanslarını kullanır.

User prompt: Bu Resource mimarisini altıgen (hex) harita gridi üzerindeki binalarla nasıl konuştururum? Üretim döngüsü komşuları nasıl kontrol eder?

Response: Godot 4'te altıgen (hex) haritalarla çalışmanın en güzel yanı, karmaşık eksenel (axial) veya küp (cube) koordinat matematiklerini sıfırdan yazmak zorunda olmamanızdır. Godot'un yerleşik `TileMap` (veya Godot 4.3+ ile `TileMapLayer`) düğümü, hex gridlerin komşuluk mantığını yerel olarak çözer.

Bu sistemi, hazırladığımız Resource mimarisiyle konuşturmak için bir **Dünya Yöneticisi (Grid/World Manager)** ve binaların içindeki **Üretim Döngüsü (Tick Loop)** mekanizmasını kurmamız gerekir.

İşte bu lojistik ve üretim döngüsünün nasıl çalıştığı:

1. Dünya Yöneticisi (GridManager) Kurulumu

Oyun alanındaki binalar birbirlerini doğrudan göremez. Haritada kimin nerede olduğunu bilen tek bir otorite olmalıdır: `GridManager`. Bu bir `Autoload` (Singleton) veya ana sahne betiği olabilir.

```
# grid_manager.gd
extends Node

# Sahnedeki TileMap referansımız
@export var hex_tilemap: TileMapLayer

# Grid üzerindeki konum (Vector2i) -> Bina Nesnesi (BuildingInstance) haritalaması
var active_buildings: Dictionary = {}

# Yeni bina eklendiğinde çağrılır
func register_building(grid_pos: Vector2i, building: Node):
    active_buildings[grid_pos] = building

# Bir hücrenin etrafındaki (6 komşu) binaları bulur
func get_neighboring_buildings(grid_pos: Vector2i) -> Array[Node]:
    var neighbors: Array[Node] = []

    # GODOT 4 BÜYÜSÜ: TileMap, hex türünüze göre etraftaki 6 hücreyi otomatik hesaplar
    var surrounding_cells = hex_tilemap.get_surrounding_cells(grid_pos)

    for cell in surrounding_cells:
        if active_buildings.has(cell):
            neighbors.append(active_buildings[cell])

    return neighbors
```

Sistem Notu: Yukarıdaki etkileşimli ızgarada bir tüketici binanın, üretici binaya temas etmediğinde üretim döngüsünü nasıl durdurduğunu görebilirsiniz. İşin GDScript tarafı tam olarak bu mantıkla çalışır.

2. Üretim Döngüsü (BuildingInstance)

Sahneye yerleştirdiğiniz her bina (Örneğin Değirmen), belirli aralıklarla (Timer ile veya global bir Tick sistemiyle) çalışır. Reçetesini okur, GridManager'a komşularını sorar ve girdi arar.

```
# building_instance.gd
extends Node2D
class_name BuildingInstance
```

```
@export var building_data: BuildingData # Resource dosyamız
var grid_position: Vector2i
var inventory: Dictionary = {} # Üretilenleri tutmak için
```

```
func _ready():
    # Binayı dünya yöneticisine kaydet
    GridManager.register_building(grid_position, self)

    # Her 2 saniyede bir üretim dene (Basit yaklaşım)
    var timer = Timer.new()
    timer.wait_time = 2.0
    timer.timeout.connect(_on_tick)
    add_child(timer)
    timer.start()

func _on_tick():
    # Bu binanın geçerli reçetesini al (şimdilik ilkini varsayalım)
    if building_data.available_recipes.is_empty(): return
    var recipe = building_data.available_recipes[0]

    # Hammadde gerekmiyorsa (Çiftlik veya Oduncu gibi) direkt üret
    if recipe.inputs.is_empty():
        produce(recipe)
        return

    # Girdiye ihtiyaç varsa komşuları kontrol et
    var neighbors = GridManager.get_neighboring_buildings(grid_position)
    var can_produce = check_inputs_from_neighbors(recipe, neighbors)

    if can_produce:
        consume_and_produce(recipe, neighbors)

# Komşularda yeterli hammadde var mı?
func check_inputs_from_neighbors(recipe: RecipeData, neighbors: Array) -> bool:
    for i in range(recipe.inputs.size()):
        var required_item = recipe.inputs[i]
        var required_amount = recipe.input_amounts[i]
        var found_amount = 0

        # 6 komşuya bak, içlerinde bu eşyayı üreten/barındıran var mı?
        for neighbor in neighbors:
            if neighbor.inventory.has(required_item):
                found_amount += neighbor.inventory[required_item]

        if found_amount < required_amount:
            return false # Bir hammadde bile eksikse üretim durur

    return true

# Eşyaları komşulardan sil ve kendi envanterine yeni eşyayı ekle
func consume_and_produce(recipe: RecipeData, neighbors: Array):
    # Gerçek bir oyunda burada komşunun envanterinden eksiltme mantığı yazılır
    # ...

    # Kendi envanterime çıktıyı ekle
    if not inventory.has(recipe.output_item):
        inventory[recipe.output_item] = 0
```

```
inventory[recipe.output_item] += recipe.output_amount  
  
print(building_data.building_name + " üretti: " + recipe.output_item.display_name)
```

Özetle Mimarinin Akışı:

1. BuildingData (Resource), binanın kapasitesini ve reęetelerini (RecipeData) tanımlar.
 2. TileMap, hex grid'in karmaşık koordinat matematiđini gizler ve size temiz bir Vector2i dizisi verir.
 3. GridManager, bu koordinatlarda kimin oturduđunun haritasını (Dictionary) tutar.
 4. Bina kendi "Tick" sinyalinde reęetesindeki girdileri okur (ItemData) ve komşularının envanterini sorgular. Eđer eşleşme varsa, işlem gerçekleşir.
-